

컴퓨터 그래픽스 기말시험 핵심 정리 (최종 완성본)

이 문서는 code 폴더의 실습 코드와 docs 폴더의 이론 문서를 종합하여, 서술형 기말시험 대비를 위해 핵심 개념 위주로 정리한 자료입니다. 각 이론에 코드 예제를 추가하고, 코드 분석을 통한 심화 학습 내용과 모든 예상 문제를 포함했습니다.

목차

- [1. OpenGL 렌더링 파이프라인](#)
- [2. 좌표계와 변환](#)
- [3. 은면 제거 \(Hidden Surface Removal\)](#)
- [4. 폴리곤 렌더링과 셰이딩](#)
- [5. 주요 함수 및 개념 요약](#)
- [6. 코드 분석 기반 심화 학습](#)
- [7. 시험 대비 예상 문제 \(종합\)](#)

1. OpenGL 렌더링 파이프라인

3D 모델 데이터(점)가 화면의 2D 픽셀로 변환되기까지의 전체 과정을 의미합니다.

GLUT 프로그램 기본 구조

초기화 -> 콜백 함수 등록 -> 메인 루프 시작 의 흐름을 가집니다.

code/2-2-3.cpp 의 main 함수 예시:

```
int main(int argc, char** argv) {
    // 1. 초기화 단계
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); // 디스플레이 모드 설정
    glutInitWindowSize(500, 500); // 윈도우 크기 설정
    glutCreateWindow("Window Title"); // 윈도우 생성

    // 2. 콜백 함수 등록
    glutDisplayFunc(RenderScene); // 화면을 그려야 할 때 호출될 함수 지정
    glutReshapeFunc(ChangeSize); // 창 크기가 바뀔 때 호출될 함수 지정

    // 3. 메인 루프 시작
    glutMainLoop(); // 이벤트 루프를 시작. 이 뒤의 코드는 실행되지 않음.
    return 0;
}
```

• 핵심 콜백 함수:

- RenderScene() (Display Callback):** 창을 다시 그려야 할 때(glutPostRedisplay() 호출 등) 운영체제에 의해 호출됩니다. glClear 로 버퍼를 지우고, glFlush 나 glutSwapBuffers 로 그리기를 완료해야 합니다.
- ChangeSize() (Reshape Callback):** 창 크기가 변경될 때 자동으로 호출됩니다. 주로 glViewport 와 투영 행렬 (glOrtho , gluPerspective)을 여기서 설정하여 종횡비를 맞춥니다.
- SpecialKeys() / Keyboard() :** 키보드 입력이 발생했을 때 호출됩니다. 이를 통해 사용자와의 상호작용을 구현할 수 있습니다.

2. 좌표계와 변환 (상세)

컴퓨터 그래픽스에서 3D 객체를 2D 화면에 표시하기까지의 과정은 여러 단계의 기하학적 변환(Geometry Transformation)으로 이루어집니다. 이 전체 과정을 변환 파이프라인(Transformation Pipeline)이라고 부릅니다.

2.1 기본 변환 (Basic Transformations)

3D 모델을 움직이고, 회전하고, 크기를 조절하는 가장 기본적인 변환입니다.

- 이동 (Translation): 객체의 위치를 특정 방향으로 옮깁니다.
- 회전 (Rotation): 객체를 특정 축을 기준으로 회전시킵니다.
- 크기 조절 (Scaling): 객체의 크기를 늘리거나 줄입니다.

이러한 변환들은 행렬 연산을 통해 이루어지는데, 이동 변환은 덧셈, 회전과 크기 조절은 곱셈으로 표현됩니다. 연산 방식이 달라 복잡하기 때문에 동차좌표계(Homogeneous Coordinates)를 도입하여 모든 변환을 행렬 곱셈으로 통일합니다.

2.2 동차좌표계 (Homogeneous Coordinates)

동차좌표계는 3D 좌표(x, y, z)에 w라는 네 번째 차원을 추가하여 (x, y, z, w) 형태로 표현하는 방식입니다. 보통 3D 공간의 점은 w=1로, 방향을 나타내는 벡터는 w=0으로 설정합니다.

동차좌표계를 사용하는 이유

1. 변환의 통일: 이동, 회전, 크기 조절 등 모든 변환을 4x4 행렬의 곱셈 하나로 표현할 수 있습니다. 이를 통해 복잡한 변환 순서를 간단하게 처리하고, 여러 변환을 미리 계산하여 하나의 행렬로 만들어 효율을 높일 수 있습니다.
2. 원근 투영 표현: 원근법에서 가까운 물체는 크게, 먼 물체는 작게 보이는 효과를 행렬 곱셈으로 자연스럽게 구현할 수 있습니다. w 값을 조절하여 원근감을 표현합니다.
3. 소실점(Vanishing Point) 표현: 무한히 먼 점(소실점)을 w=0으로 설정하여 수학적으로 명확하게 다룰 수 있습니다.

2.3 변환 파이프라인 (Transformation Pipeline)

3D 모델 데이터(정점)가 화면에 그려지기까지 거치는 변환 과정입니다.

로컬 공간 -> 월드 공간 -> 뷰(카메라) 공간 -> 클립 공간 -> NDC -> 화면 공간

1) 모델 변환 (Model Transformation)

- 로컬 공간(Local Space)에 있는 모델을 월드 공간(World Space)으로 배치하는 과정입니다.
- 각 객체는 자신만의 좌표계(로컬 공간)를 가집니다. 이 객체들을 월드 공간이라는 하나의 공통된 공간에 배치하기 위해 이동, 회전, 크기 조절을 적용합니다.

2) 관측 변환 (Viewing Transformation)

- 월드 공간의 객체들을 뷰(카메라) 공간(View Space)으로 변환합니다.
- 카메라의 위치와 방향을 기준으로 월드 전체를 재배치하는 것과 같습니다. 즉, 카메라가 원점(0,0,0)에 위치하고 -Z축을 바라보도록 월드 공간의 모든 객체를 함께 움직이는 변환입니다.

3) 모델뷰 변환 (Model-View Transformation)

- 모델 변환과 관측 변환은 종종 합쳐져 모델뷰 행렬(Model-View Matrix)이라는 하나의 행렬로 만들어집니다.
- 모델뷰의 이중성: 모델뷰 변환은 두 가지 관점으로 해석될 수 있습니다.
 1. 모델링 변환 + 관측 변환: 객체를 월드 공간에 배치하고(모델링), 이 월드를 카메라 시점으로 가져온다(관측).
 2. 고정된 카메라와 움직이는 객체: 카메라를 원점에 고정시키고, 객체를 카메라 앞으로 가져와 특정 위치에 배치하는 것으로 해석할 수도 있습니다.
- 결과적으로는 동일하며, 두 변환을 합쳐서 연산하면 효율적입니다.

4) 투영 변환 (Projection Transformation)

- 뷰 공간에 있는 3D 모델을 2D로 표현하기 위해 클립 공간(Clip Space)으로 변환하는 과정입니다. 카메라의 절두체(Frustum) 내에 보이는 영역을 정규화된 공간으로 찌그러뜨립니다.

- 원근 투영 (**Perspective Projection**): 가까운 물체는 크게, 먼 물체는 작게 보이게 하여 원근감을 줍니다. 일반적인 3D 게임이나 시뮬레이션에서 사용됩니다.
- 직교 투영 (**Orthographic Projection**): 거리에 상관없이 모든 물체를 같은 크기로 표현합니다. 건축 도면이나 CAD, 일부 2D 스타일 게임에서 사용됩니다.

5) 뷰포트 변환 (Viewport Transformation)

- 투영 변환을 거친 클립 공간의 좌표는 보통 -1에서 1 사이의 정규화된 장치 좌표(**Normalized Device Coordinates, NDC**)로 변환됩니다.
- 이 NDC 좌표를 실제 화면(윈도우)의 특정 영역, 즉 뷰포트(**Viewport**)의 픽셀 좌표로 최종 변환합니다.
- 일반적으로는 NDC와 뷰포트가 1:1로 대응되지만, 화면 분할(Split Screen)과 같이 하나의 화면에 여러 뷰를 보여줄 때는 특정 영역에만 렌더링하도록 뷰포트 설정을 변경할 수 있습니다.

2.4 추가 주요 개념

버텍스(Vertex)와 포인트(Point)의 차이

- 포인트(**Point**): 단순히 공간상의 한 위치(좌표)를 의미합니다.
- 버텍스(**Vertex**): 그래픽스에서 사용되는 더 구체적인 개념으로, 위치 정보(좌표)뿐만 아니라 색상(**Color**), 법선 벡터(**Normal Vector**), 텍스처 좌표(**Texture Coordinates**) 등 렌더링에 필요한 추가 데이터를 포함하는 '정점'을 의미합니다. 폴리곤(Polygon)과 같은 기하학적 도형을 구성하는 기본 단위입니다.

안티에일리어싱 (Anti-aliasing)

- 디지털 화면은 픽셀이라는 사각형 격자로 이루어져 있어 직선이나 곡선을 표현할 때 계단 현상(에일리어싱, **Aliasing**)이 발생합니다.
- 안티에일리어싱은 이 계단 현상을 완화하는 기술입니다. 경계선 주변의 픽셀 색상을 배경색과 물체색의 중간 값으로 채워 경계를 부드럽게 보이게 만듭니다.

행렬 스택 (Matrix Stack): Push와 Pop

복잡한 장면을 렌더링할 때, 여러 객체가 계층적으로 변환되거나(예: 태양계 모델에서 행성이 태양 주위를 돌고, 위성이 행성 주위를 도는 경우), 특정 변환 상태를 임시로 저장하고 복원해야 할 때가 있습니다. 이때 행렬 스택(**Matrix Stack**)이 유용하게 사용됩니다.

- Push**: 현재 변환 행렬(예: 모델뷰 행렬)을 스택의 상단에 저장합니다. 이를 통해 현재의 변환 상태를 '백업'해 둘 수 있습니다.
- Pop**: 스택의 가장 상단에 저장된 행렬을 가져와 현재 변환 행렬로 설정합니다. 이를 통해 이전에 저장했던 변환 상태로 '복원'할 수 있습니다.

3. 은면 제거 (Hidden Surface Removal)

보이지 않는 부분을 그리지 않아 렌더링 성능을 향상시키는 핵심 기술입니다.

1) 깊이 버퍼 (Z-Buffer)

픽셀 단위로 깊이(거리)를 비교하여 가까운 물체만 그립니다.

- 사용법:
 - `glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);` : main 에서 깊이 버퍼 요청
 - `glEnable(GL_DEPTH_TEST);` : 렌더링 설정에서 깊이 테스트 활성화
 - `glClearColor(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);` : 매 프레임마다 색상 버퍼와 함께 깊이 버퍼도 초기화

2) 후면 제거 (Back-face Culling)

카메라를 등진 뒷면을 아예 그리지 않는 최적화 기법입니다.

- 앞/뒷면 구분: 정점의 순서(**Winding Order**)로 구분합니다. (기본: 반시계 방향이 앞면)

- `code/3-7-1.cpp` 예시:

```
void RenderScene(void) {
    glClear(GL_COLOR_BUFFER_BIT);

    // 후면 제거 활성화
    glEnable(GL_CULL_FACE);

    // [원뿔 그리는 코드]
    // ...
}
```

이 코드에서 원뿔을 회전시킬 때, 뒷면은 그려지지 않아 렌더링 부하가 줄어듭니다. 만약 `glEnable(GL_CULL_FACE)` 를 주석 처리하면, 원뿔의 뒷면까지 모두 그려지게 되어 (투명 객체가 아닌 이상) 시각적으로는 차이가 없지만 성능은 저하됩니다.

4. 폴리곤 렌더링과 셰이딩

폴리곤 렌더링 방식

- `glPolygonMode(face, mode)` : 폴리곤을 채울지(`GL_FILL`), 선으로 그릴지(`GL_LINE`), 점으로 그릴지(`GL_POINT`) 결정합니다.
- `GL_TRIANGLE_FAN` : 중심점을 공유하는 부채꼴 모양 삼각형들을 그립니다. `code/3-7-1.cpp` 에서 원뿔의 옆면과 밑면을 그릴 때 사용되었습니다.

셰이딩 모델

- `glShadeModel(mode)` : 폴리곤 내부 색상을 어떻게 채울지 결정합니다.
- 플랫 셰이딩 (`GL_FLAT`) : 폴리곤 전체를 단색으로 칠합니다. 각진 느낌을 주며, 폴리곤의 마지막 정점 색상을 사용합니다.

`code/3-7-1.cpp` 예시:

```
int main(...) {
    // ...
    glShadeModel(GL_FLAT); // 플랫 셰이딩으로 설정
    // ...
}
```

- 스무스 셰이딩 (`GL_SMOOTH` , 기본값) : 각 정점의 색상을 보간(interpolate)하여 부드러운 그라데이션 효과를 냅니다. `shademodel.cpp` 또는 유사 파일 예시:

```
glBegin(GL_TRIANGLES);
glColor3f(1.0f, 0.0f, 0.0f); // 첫 정점은 빨간색
glVertex2f(0.0f, 0.0f);

glColor3f(0.0f, 1.0f, 0.0f); // 두 번째 정점은 초록색
glVertex2f(50.0f, 0.0f);

glColor3f(0.0f, 0.0f, 1.0f); // 세 번째 정점은 파란색
glVertex2f(50.0f, 50.0f);
glEnd();
```

`GL_SMOOTH` 모드(기본값)에서는 위 삼각형이 빨강, 초록, 파랑이 섞인 부드러운 그라데이션으로 채워집니다. 만약 `GL_FLAT` 모드였다면, 삼각형 전체가 마지막 정점의 색상인 파란색으로 칠해졌을 것입니다.

5. 주요 함수 및 개념 요약

함수/개념	역할	서술형 포인트
<code>glutInitDisplayMode()</code>	디스플레이 모드(버퍼 종류, 색상 모델) 설정	GLUT_SINGLE / GLUT_DOUBLE , GLUT_RGB , GLUT_DEPTH 등 조합.
<code>glutPostRedisplay()</code>	시스템에 다음 프레임에 화면을 다시 그려달라고 요청	애니메이션 등에서 <code>RenderScene</code> 을 강제로 호출하게 만듦.
<code>glPushMatrix()</code>	현재 행렬을 스택에 복사하여 저장	복잡한 변환 전 상태를 백업.
<code>glPopMatrix()</code>	스택 최상단의 행렬을 현재 행렬로 복원	백업한 상태로 되돌아감.
<code>glColor3f()</code>	현재 그리기 색상을 설정 (float, 0.0~1.0)	상태 기계의 일부로, 한 번 설정하면 계속 유지됨.
<code>glVertex3f()</code>	3D 공간의 정점 하나를 지정	<code>glBegin / glEnd</code> 블록 안에서만 의미가 있음.

6. 코드 분석 기반 심화 학습

실습 코드들의 주석과 구조에서 발견된 시험 출제 가능성이 높은 핵심 개념들입니다.

가. 행렬 스택과 변환 상태 관리 (`code/4-2-a.cpp` vs `code/4-2.cpp`)

`glPushMatrix()` 와 `glPopMatrix()` 의 역할은 변환의 상태를 저장하고 복원하는 것입니다. 두 코드의 차이로 이 개념을 이해할 수 있습니다.

- `code/4-2-a.cpp` (정상적인 경우):

```
// 첫 번째 구
glPushMatrix(); // ✓ 현재 상태 (카메라 변환만 적용됨) 저장
    glTranslatef(0.0f, 10.0f, 0.0f);
    glutSolidSphere(1.0f, 30, 30);
glPopMatrix(); // ✓ 저장했던 상태로 복원. glTranslatef(0,10,0)은 취소됨.

// 두 번째 구
glPushMatrix();
    glTranslatef(10.0f, 0.0f, 0.0f);
    glutSolidSphere(1.0f, 30, 30);
glPopMatrix();
```

결과: 두 구는 각각 독립적인 `glTranslatef` 의 영향을 받습니다. `glPopMatrix` 덕분에 두 번째 구는 첫 번째 구의 이동에 영향을 받지 않고, 원래의 카메라 시점에서 새롭게 이동합니다.

- `code/4-2.cpp` (잘못된 경우):

```
// 첫 번째 구
glTranslatef(0.0f, 10.0f, 0.0f);
glutSolidSphere(1.0f, 30, 30);

// glLoadIdentity() 호출로 모델뷰 행렬 재설정
glLoadIdentity(); // ✗ 카메라 변환까지 모두 초기화됨!

// 두 번째 구
glTranslatef(10.0f, 0.0f, 0.0f);
glutSolidSphere(1.0f, 30, 30);
```

결과: `glLoadIdentity()` 가 모델뷰 행렬 전체를 단위행렬로 만들어 버리므로, 처음에 설정했던 카메라 변환 (`glTranslatef(0,0,-30)` 등)이 사라집니다. 따라서 두 번째 구는 전혀 다른 위치(카메라 변환이 없는 기본 시점)에 그려지게 됩니다.

나. 이벤트 처리와 화면 갱신 (code/3-7-0-a.cpp , code/4-3-0.cpp)

사용자 입력(KeyboardFunc)이나 시간 경과(TimerFunc)에 따라 화면이 바뀌는 애니메이션을 구현할 때, glutPostRedisplay() 호출은 필수적입니다.

- code/3-7-0-a.cpp 의 Keyboard 함수:

```
void Keyboard(unsigned char key, int x, int y) {
    if(key == 'a') {
        xTran -= 2.0f; // 객체의 x좌표 값만 변경
    }
    // ...
    glutPostRedisplay(); // ★ 중요 포인트 ★ 화면을 다시 그려달라고 요청
}
```

glutPostRedisplay() 는 "다음 프레임에 RenderScene 함수를 호출해달라"고 GLUT에 요청하는 역할을 합니다. 만약 이 호출이 없다면, xTran 변수값은 바뀌지만 RenderScene 이 다시 호출되지 않아 화면에는 아무런 변화가 없습니다.

- code/4-3-0.cpp 의 TimerFunc 함수:

```
void TimerFunction(int value) {
    colorIndex = (colorIndex + 1) % 3; // 색인 변경
    glutPostRedisplay(); // 화면 갱신 요청
    glutTimerFunc(1000, TimerFunction, 1); // 1초 뒤 자기 자신을 다시 호출
}
```

주기적으로 상태를 바꾸는 애니메이션도 마찬가지입니다. 상태 변경 후 glutPostRedisplay() 를 호출해야만 실제 화면에 반영됩니다.

다. glOrtho 의 깊이 방향 반전 (code/3-7-a-1.cpp)

glOrtho(left, right, bottom, top, near, far) 에서 near 와 far 값의 순서는 깊이 방향을 결정합니다.

- 일반적인 경우: near < far (예: near=-100, far=100)
 - z값이 클수록 카메라에서 멀어집니다.
- *** 중요: glOrtho에서 near > far인 경우 ***: near > far (예: near=100, far=-100)
 - 깊이 방향이 반전됩니다. z값이 클수록 오히려 카메라에 가깝다고 판단됩니다.
 - 이것은 깊이 테스트(Z-buffering)의 결과에 직접적인 영향을 미치므로, 시험에서 함정 문제로 출제되기 좋은 주제입니다.

라. 선과 점의 속성 변경 (code/3-1-4.cpp , code/3-5-1.cpp)

glBegin 밖에서 속성 함수를 호출하여 앞으로 그려질 선과 점의 모양을 제어할 수 있습니다.

- 점선 그리기 (glLineStipple): glEnable(GL_LINE_STIPPLE) 로 활성화하며, glLineStipple(factor, pattern) 으로 모양을 지정합니다.
 - factor: 패턴의 각 비트를 몇 번 반복할지 결정. 클수록 점선 간격이 넓어집니다.
 - pattern: 16비트 마스크. 비트가 1인 부분은 그려지고 0인 부분은 그려지지 않습니다. (예: 0x5555 = 0101..., 0x00FF = ...000011111111)
- 점 크기 (glPointSize): glPointSize(size) 는 점의 크기를 픽셀 단위로 지정합니다.

7. 시험 대비 예상 문제 (종합)

파트 1: 핵심 서술형 문제

- ㉠ Q1. OpenGL 렌더링 파이프라인의 주요 단계를 설명하고, GL_MODELVIEW 행렬과 GL_PROJECTION 행렬이 각각 어느 단계에서 사용되는지 설명하십시오.

③ 상세 풀이 >

렌더링 파이프라인은 정점 데이터가 화면 픽셀로 변환되는 과정입니다. 주요 단계는 로컬 공간 -> 월드 공간 -> 뷰 공간 -> 클립 공간 -> NDC -> 화면 공간 순입니다. `GL_MODELVIEW` 행렬은 모델 변환(로컬->월드)과 뷰 변환(월드->뷰)에 사용되며, `GL_PROJECTION` 행렬은 뷰 공간에서 클립 공간으로 변환하는 투영 변환에 사용됩니다.

- ② Q2. `glViewport` 함수와 `glOrtho` 함수의 역할을 비교 설명하고, 창 크기가 변할 때 이 두 함수가 어떻게 함께 사용되어야 하는지 설명하시오.

③ 상세 풀이 >

`glOrtho` 는 3D 공간에 그려질 논리적인 좌표계의 범위를 설정하는 투영 변환입니다. 반면 `glViewport` 는 이 논리적 공간이 매핑될 윈도우 상의 실제 픽셀 영역을 지정하는 뷰포트 변환입니다. 창 크기가 변하면, 먼저 `glViewport` 를 새 창 크기에 맞게 설정하고, 그 다음 `glOrtho` 의 범위를 창의 종횡비에 맞게 조절하여 내용물이 찌그러지지 않게 해야 합니다.

- ③ Q3. 은면 제거 기법인 '깊이 버퍼(Z-Buffer)'와 '후면 제거(Back-face Culling)'의 원리를 각각 설명하고, 어떤 차이가 있는지 설명하시오.

③ 상세 풀이 >

깊이 버퍼는 픽셀 단위로 카메라와의 거리를 비교하여 가려진 면을 제거하는 기법입니다. 픽셀을 그릴 때 기존 픽셀보다 카메라에 더 가까울 때만 덮어씁니다. 후면 제거는 폴리곤의 법선 벡터가 카메라를 등지고 있을 경우, 즉 뒷면일 경우 해당 폴리곤을 아예 렌더링 과정에서 제외하는 기법입니다. 깊이 버퍼는 시점과 무관하게 겹치는 모든 물체에 적용되지만, 후면 제거는 닫힌 객체의 뒷면을 그리지 않는 최적화에 중점을 둔다는 차이가 있습니다.

- ③ Q4. 플랫 셰이딩(Flat Shading)과 스무스 셰이딩(Smooth Shading)의 차이점을 설명하고, OpenGL에서 각각 어떻게 설정하는지 설명하시오.

③ 상세 풀이 >

플랫 셰이딩은 폴리곤 전체를 하나의 색상으로 칠해 각진 느낌을 줍니다. OpenGL에서는 `glShadeModel(GL_FLAT)` 으로 설정합니다. 반면 스무스 셰이딩은 각 정점의 색상을 보간하여 폴리곤 내부에 부드러운 색상 전환(그라데이션)을 만들어 사실적인 느낌을 줍니다. `glShadeModel(GL_SMOOTH)` 로 설정하며, 이것이 기본값입니다.

- ③ ☆ Q5. (교수님 강조 문제) 다음 조건에서 깊이 테스트와 후면 제거가 모두 활성화되어 있고, 두 원 $A(z=10)$, $B(z=20)$ 가 그려질 때, 최종적으로 화면에 무엇이 보일지 그 이유와 함께 서술하시오. (단, 카메라 위치는 원점, 시선은 -Z 방향)

③ 상세 풀이 >

정답: "알 수 없다"

이유 상세 설명:

1. 단순 깊이 비교의 함정: `glEnable(GL_DEPTH_TEST)` 가 활성화되어 있으므로 카메라에 더 가까운 물체가 그려져야 합니다. 카메라($z=0$)에서 볼 때 원 $A(z=10)$ 가 원 $B(z=20)$ 보다 가까우므로, 원 A 가 보일 것이라고 생각하기 쉽습니다.

2. **후면 제거의 결정적 역할:** 하지만 `glEnable(GL_CULL_FACE)` 가 활성화되어 있기 때문에, 깊이 비교 이전에 각 원이 '앞면'인지 '뒷면'인지 판별하는 과정이 먼저 일어납니다. 뒷면으로 판정된 원은 렌더링 대상에서 아예 제외됩니다.
3. **앞/뒷면 판별 기준 부재:** 폴리곤의 앞/뒷면은 정점을 그리는 순서(Winding Order)에 따라 결정됩니다. OpenGL은 기본적으로 반시계 방향(CCW)을 앞면으로 봅니다. XY 평면의 원을 카메라(-Z 방향)가 볼 때, 원의 정점들이 시계 방향(CW)으로 그려져야 법선 벡터가 카메라를 향해 앞면으로 인식됩니다. 반시계 방향(CCW)으로 그려지면 법선 벡터가 카메라 반대 방향을 향해 뒷면으로 인식되고, 후면 제거에 의해 그려지지 않습니다.
4. **시나리오 분석:**
 - 시나리오 1: 두 원 모두 앞면(CW)으로 그려졌다면, 깊이 테스트에 의해 더 가까운 원 **A**만 보입니다.
 - 시나리오 2: 두 원 모두 뒷면(CCW)으로 그려졌다면, 둘 다 후면 제거되어 아무것도 보이지 않습니다.
 - 시나리오 3: 원 A는 뒷면(CCW), 원 B는 앞면(CW)으로 그려졌다면, 원 A는 제거되고 원 **B**만 보입니다.
 - 시나리오 4: 원 A는 앞면(CW), 원 B는 뒷면(CCW)으로 그려졌다면, 원 B는 제거되고 원 **A**만 보입니다.
5. **결론:** 문제에서 두 원의 정점 순서(Winding Order)에 대한 정보가 주어지지 않았기 때문에, 각 원이 앞면인지 뒷면인지 확인할 수 없습니다. 따라서 위 시나리오 중 어떤 결과가 나올지 알 수 없으므로, 정답은 "알 수 없다"입니다. 만약 후면 제거가 비활성화되어 있다면, 정답은 "원 A가 보인다"가 됩니다.

파트 2: 응용 및 단답형 문제

- ② **Q6. (응용/코드)** `code/3-1.cpp` 파일에서 `ChangeSize` 함수의 `glOrtho` 호출 부분을 `glOrtho(-100, 100, -100, 100, -1, 1);` 로 수정하면 어떤 결과가 나타날지, 그 이유와 함께 서술하시오.

③ 상세 풀이 >

점 `(50.0f, 50.0f, 50.0f)` 가 화면에 보이지 않게 됩니다. 그 이유는 `glOrtho` 의 마지막 두 인자 `zNear` 와 `zFar` 가 시야의 깊이 범위를 결정하는데, 이 값이 각각 -1과 1로 설정되었기 때문입니다. 점의 z좌표인 50은 이 범위(-1 ~ 1)를 벗어나므로, 클리핑(clipping)되어 렌더링되지 않습니다.

- ② **Q7. (단순 암기)** OpenGL에서 폴리곤의 앞면과 뒷면을 구분하는 기준은 무엇이며, 기본적으로 어느 쪽이 앞면으로 간주되니까?

③ 상세 풀이 >

정점의 순서(Winding Order)로 구분하며, 기본적으로 반시계 방향(Counter-Clockwise, CCW)이 앞면으로 간주됩니다.

- ② **Q8. (응용/코드)** `code/2-2-3.cpp` 의 `RenderScene` 함수를 수정하여, 빨간색 사각형 대신 파란색 원을 그리고자 합니다. 어떤 코드를 작성해야 할지 `glBegin/glEnd` 를 사용하여 설명하시오.

③ 상세 풀이 >

먼저 `glColor3f(0.0f, 0.0f, 1.0f);` 로 색상을 파란색으로 변경해야 합니다. 그 다음 `GL_TRIANGLE_FAN` 을 사용하여 원을 그립니다. 원의 중심점을 첫 정점으로 지정하고, `for` 루프를 돌며 `cos` 와 `sin` 함수를 이용해 원주 위의 점들을 계산하여 `glVertex2f` 로 추가합니다.

```
glColor3f(0.0f, 0.0f, 1.0f); // 파란색
glBegin(GL_TRIANGLE_FAN);
glVertex2f(0.0, 0.0); // 원의 중심
for(int i = 0; i <= 360; i++) {
    float angle = i * 3.14159f / 180.0f;
    float x = 50.0f * cos(angle);
```

```
float y = 50.0f * sin(angle);
glVertex2f(x, y);
}
glEnd();
```

② Q9. (문제 해결) `glEnable(GL_CULL_FACE)` 를 사용하여 후면 제거를 활성화했습니다. 큐브를 `GL_TRIANGLES` 로 그리고 회전시켰더니, 뒷면들은 잘 사라지는데 유독 한쪽 면만 항상 보이지 않습니다. 가장 가능성 있는 원인은 무엇입니까?

② 상세 풀이 >

해당 면을 구성하는 점점들의 순서(Winding Order)가 다른 면들과 반대로(예: 시계 방향으로) 정의되었을 가능성이 높습니다. 후면 제거는 점점 순서를 기준으로 앞/뒷면을 판단하는데, 순서가 반대이면 항상 뒷면으로 간주되어 컬링(culling)되기 때문입니다.

② Q10. (단답형) 이중 버퍼링(Double Buffering)을 사용할 때, 화면 깜빡임(flickering)을 방지하기 위해 `glFlush()` 대신 호출해야 하는 함수는 무엇입니까?

② 상세 풀이 >

```
glutSwapBuffers()
```

② Q11. (개념) `glPushMatrix()` 와 `glPopMatrix()` 를 사용하는 주된 이유는 무엇이며, 어떤 상황에서 유용합니까?

② 상세 풀이 >

현재 적용된 변환 행렬(상태)을 임시로 저장하고 나중에 복원하기 위해 사용합니다. 태양계 모델처럼, 특정 객체(행성)에 적용된 변환이 다른 객체(위성)에 영향을 주지 않도록 하거나, 복잡한 계층 구조를 가진 모델을 렌더링할 때 매우 유용합니다. 행성의 변환을 적용하기 전에 `glPushMatrix()` 로 태양계의 원점 상태를 저장하고, 행성 및 위성 렌더링이 끝난 후 `glPopMatrix()` 로 복원하여 다른 행성을 그릴 준비를 할 수 있습니다.

② Q12. (코드 분석) `code/4-2.cpp` 에서 두 번째 구를 그리기 직전에 `glLoadIdentity()` 가 호출됩니다. 이로 인해 두 번째 구는 첫 번째 구와 어떻게 다르게 보일지, 그 이유를 카메라 변환과 연관지어 설명하십시오.

② 상세 풀이 >

첫 번째 구는 `ChangeSize` 에서 설정한 카메라 변환(`glTranslatef(0,0,-30)`)이 적용된 상태에서 그려집니다. 하지만 두 번째 구를 그리기 전 `glLoadIdentity()` 가 모델뷰 행렬을 단위행렬로 초기화시키면서 이전에 적용된 카메라 변환까지 모두 사라지게 됩니다. 따라서 두 번째 구는 카메라 변환이 없는 기본 시점(카메라가 원점에 있는 상태)을 기준으로 그려지므로, 첫 번째 구와 전혀 다른 위치와 크기로 보이게 됩니다.

② Q13. (개념) `glutTimerFunc` 나 `glutKeyboardFunc` 콜백 함수 내에서 `glutPostRedisplay()` 를 호출하는 이유는 무엇입니까? 만약 이 호출을 생략하면 어떤 현상이 발생합니까? (`code/4-3-0.cpp` 와 `code/3-7-0-a.cpp` 참고)

② 상세 풀이 >

콜백 함수 내에서 객체의 위치나 색상 같은 상태 변수 값을 변경하더라도, `RenderScene` 함수가 다시 호출되지 않으면 화면은 갱신되지 않습니다. `glutPostRedisplay()` 는 GLUT에게 다음 렌더링 주기에 `RenderScene` 을 호출해달라고 요청하는 역할을 합니다. 따라서 이 호출을 생략하면, 내부적으로 변수 값은 바뀌지만 화면에는 아무런 시각적 변화가 나타나지 않습니다.

② **Q14. (단답형)** `glLineStipple` 함수의 두 인자(`factor` , `pattern`)는 각각 무엇을 제어하는지 설명하시오. (`code/3-5-1.cpp` 참고)

② 상세 풀이 >

`factor` 는 `pattern` 의 각 비트(0 또는 1)를 몇 번씩 반복할지 결정하는 정수이며, 값이 클수록 점선의 간격이 넓어집니다. `pattern` 은 점선을 구성하는 16비트 마스크로, 1인 부분은 그려지고 0인 부분은 그려지지 않아 실제 점선의 모양을 결정합니다.